CS260 — Data Structures

Reading Week 2 · *Algorithms Illuminated* · Ch 1.4, 1.5, 5.1–5.3

Question 1: Reading Notes

MergeSort — Ch 1.4 (The Algorithm)

- **Divide & Conquer** → split array in half, sort each half recursively, merge results
- **Base case:** array of size ≤ 1 is already sorted
- **Merge:** scan two sorted halves left-to-right, write smaller element → O(n)
- NOT in-place → needs O(n) extra memory

MergeSort — Ch 1.5 (The Analysis)

- **Recurrence:** T(n) = 2T(n/2) + O(n)
- **Recursion tree:** $\log_2 n$ levels, O(n) work per level → Θ**(n log n)** always
- Best, average, worst case all the same → always Θ(n log n)

QuickSort — Ch 5.1 (Overview)

- Also D&C, but division does the real work
- **Steps:** choose pivot → partition → recurse on both sides
- After partition, pivot is in its **final position** — never moves again
- **In-place** (only O(log n) stack space)

QuickSort — Ch 5.2 (Partitioning)

- Rearranges array: **[< pivot | pivot | > pivot]**
- Single linear scan → **O(n)**
- Pivot index fixed after partition; recurse on left and right independently

QuickSort — Ch 5.3 (Good Pivots)

Pivot Choice	Split	Runtime
Median (best)	50/50	O(n log n)
Random	~25/75 on average	Expected O(n log n)
Min or Max (worst)	0/n	$O(n^2)$

■ Random pivot selection → good pivot ~50% of the time → guaranteed expected O(n log n)

Question 2: MergeSort

(a) Divide-and-Conquer

Break a problem into smaller subproblems of the same type, solve them recursively, then combine. Small enough subproblems become trivial (base case).

(b) Other D&C; algorithms so far?

Yes — Binary Search. It splits the search space in half each step and recurses on one side → same D&C structure.

The other 4 (Linear Search, Bubble, Selection, Insertion Sort) are iterative — no recursive subdivision.

(c) Recurrence: T(n) = 2T(n/2) + f(n)

- **a = 2** → MergeSort makes 2 recursive calls (left half + right half)
- **b = 2** → each call is on input of size n/2 (array split in half)
- **f(n) = Θ(n)** → Merge scans all n elements once, doing O(1) work per element

---

Question 3: QuickSort

(a) Partition around pivot = 5

Starting array:

1	9	2	6	7	8	11	5	3

After partition (pivot = **5** → goes to index 3):

1	2	3	**5**	9	6	7	8	11

→ 5 is now in its final sorted position

(b) Why use QuickSort over MergeSort?

Despite worst-case $O(n^2)$, QuickSort wins in practice because:

- **In-place** → O(n) less memory than MergeSort
- **Better cache performance** → works on contiguous memory, fewer cache misses
- **Smaller constants** → simpler inner loop, faster in practice
- With random pivots, $O(n^2)$ is astronomically unlikely